

Trabalho 2 - Técnicas de Programação 2 (CIC0198)

Relatório de Desenvolvimento Orientado a Testes (TDD)

Raul Myron Silva Amorim

Matrícula: 200049712

Maio de 2026

1. Teste de Caixa Fechada: Tabela de Decisão

O sistema de monitoramento de logs foi implementado mapeando regras estritas de manipulação de arquivos e sanitização de dados. A tabela de decisão a seguir consolida as regras de negócio aplicadas no fluxo principal (`process_log_list`) e na validação individual de linhas (`parse_log_line`), modeladas e guiadas pelos testes unitários no framework Catch2.

Condições (C) e Ações (A)	R1	R2	R3	R4	R5	R6
<i>Condições</i>						
C1: Arquivo de lista <code>logs.txt</code> existe?	Não	Sim	Sim	Sim	Sim	Sim
C2: Caminho de log listado existe no disco?	-	Não	Sim	Sim	Sim	Sim
C3: Log possui pelo menos uma linha válida?	-	-	Não	Não	Sim	Sim
C4: Arquivo <code>total_X.txt</code> já existe?	-	-	Não	Sim	Não	Sim
<i>Ações</i>						
A1: Retorna código de erro (-1)	X					
A2: Ignora caminho silenciosamente		X	X			
A3: Lê arquivo <code>total_X.txt</code> prévio				X		X
A4: Cria/Grava <code>total_X.txt</code> ordenado					X	
A5: Faz merge ordenado e regrava				X		X

Cobertura e Justificativa das Regras

- **R1:** Se a entrada principal falha, a execução é abortada (*Teste T13*).
- **R2:** Garante a robustez do programa caso diretórios ou arquivos tenham sido apagados externamente (*Teste T15*).

- **R3 e R4:** Evita a proliferação de arquivos totais vazios no disco se o processamento das mensagens não extrair nenhuma linha válida. O merge com arquivos em branco previne corrupção de arquivos já computados (*Teste T16 e T20*).
- **R5 e R6:** Representam o fluxo ideal (caminho feliz) de inicialização e intercalação com o histórico existente no sistema de arquivos (*Testes T17 e T18*).

2. Teste de Caixa Aberta: Expressões Regulares

A cobertura de teste de caixa aberta foi assegurada injetando a biblioteca `<regex>` em um teste dedicado (*Teste T21*). O objetivo foi provar estaticamente que os logs classificados como válidos pelo módulo de parse e posteriormente gravados em disco respeitam estritamente a especificação estrutural e a barreira de limite de memória da string.

Expressão Regular Utilizada:

```
^\d+/\d+/\d+ \d+:\d+:\d+ .{1,100}$
```

Como a Regex testa a implementação

A expressão regular avalia uma linha completa gerada pela estrutura validada:

- `^` e `$` garantem que a correspondência engloba exatamente o início e o fim da string extraída, evitando substrings residuais.
- `\d+/\d+/\d+` mapeia o separador de data configurado estaticamente em código (`kDateSep`).
- `\d+:\d+:\d+` mapeia o separador de horário configurado (`kTimeSep`).
- `.{1,100}` atua como prova matemática sobre as barreiras impostas pelas constantes `kMinMsgLen` e `kMaxMsgLen`. Mensagens vazias (0) ou truncadas/superiores (101) são detectadas na suíte de testes de regressão, assegurando que o código interno não viole requisitos limites (boundary conditions) de manipulação de memória de strings.